
Advanced System Applications and Practice for Internet and Multimedia

TM1117

Django IoT App

Instructor :

**Industrial Centre
The Hong Kong Polytechnic University**

Example: Create IoT website

- Allow people to view IoT Data (e.g. Temperature)
- Subscribe to "IoT" MQTT Topic
- Decode the IoT JSON data and store in the database



ID	Temperature	Created Date
23	24.00	May 28, 2021, 3:35 p.m.
22	24.00	May 28, 2021, 3:35 p.m.
21	24.00	May 28, 2021, 3:35 p.m.
20	24.00	May 28, 2021, 3:35 p.m.
19	24.00	May 28, 2021, 3:35 p.m.
18	24.00	May 28, 2021, 3:35 p.m.
17	24.00	May 28, 2021, 3:34 p.m.

Create "iot" application(app)

- 1) Shutdown the django server in the Anaconda Prompt
 - a) Press CTRL + C
- 2) Create "iot" application(app)
\$ python manage.py startapp iot
- 3) Register the "iot" app to the project in Visual Studio Code
 - a) Open the mysite\mysite\settings.py file in the Visual Studio Code
 - b) Search the "INSTALLED_APPS" location
 - c) Add the "iot" app to "INSTALLED_APPS" variable

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'blog',  
    'iot',  
]
```



Create "Event" Model

- 1) Create the model object "Event" in the [mysite/iot/models.py](#)
 - a) Open the [models.py](#) in Visual Studio Code
 - b) Create a table "Event" and 4 attributes "**node_id**", "**node_loc**", "**temp**" and "**date_created**"
 - c) Create a **__str__()** method to represent the object or model

```
# Create your models here.
class Event(models.Model):
    #Fields
    node_id = models.CharField(max_length=2)
    node_loc = models.CharField(max_length=10)
    temp = models.DecimalField(max_digits=5, decimal_places=2)
    date_created = models.DateTimeField(auto_now_add=True)

    #Methods
    def __str__(self):
        return 'Event {}'.format(self.id)
```

Register "Event" Model

- 1) Register the model object "Event" in the [mysite/iot/admin.py](#)
 - a) Open the [admin.py](#) in Visual Studio Code
 - b) Import the Event package (**Brown arrow**) at the beginning of the admin.py



```
from django.contrib import admin  
from .models import Event
```

- c) Register the Event model in the [admin.py](#) after the import statements

```
# Register your models here.  
admin.site.register(Event)
```

Add "Event" Model to the database

- 1) Synchronize the model into the database by running migrations to reflect the changes in the database

- a) Go to the Anaconda Prompt

\$ python manage.py makemigrations

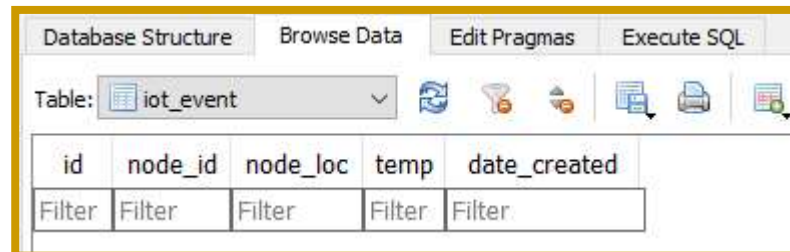
```
(django-env) (base) D:\01-Home\workspace\IC2140\01-Blog\mysite>python manage.py makemigrations
Connected with result code 0
Migrations for 'blog':
  blog\migrations\0002_auto_20210514_1501.py
    - Change Meta options on entry
Migrations for 'iot':
  iot\migrations\0001_initial.py
    - Create model Event
```

\$ python manage.py migrate

```
(django-env) (base) D:\01-Home\workspace\IC2140\01-Blog\mysite>python manage.py migrate
Connected with result code 0
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, iot, sessions
Running migrations:
  Applying blog.0002_auto_20210514_1501... OK
  Applying iot.0001_initial... OK
```

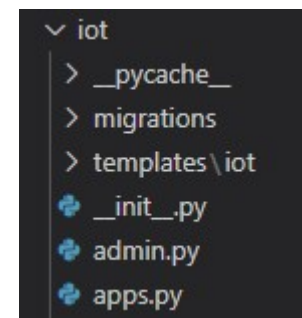
Check the "Event" model in SQLite Database Browser

- Check out the "Event" model in the database browser
 - a) Open the software "SQLite Database Browser"
 - b) Click "Open Database" and select "db.sqlite3" under the [D:\01-Home\workspace\IC2140\01-Blog\mysite](#) folder
 - c) Click the "Browser Data" Tab
 - d) Select "iot_event" as the Table



Create IoT index page (home page)

- 1) Create a folder "templates" under the `mysite/iot` folder in the Visual Studio Code
 - a) Right-click the "iot" folder in Visual Studio Code
 - b) Select the "New folder" and assign a name "templates" to it
- 2) Create a folder "iot" under the `mysite/iot/templates` folder in the Visual Studio Code
 - a) Right-click the "templates" folder in Visual Studio Code
 - b) Select the "New folder" and assign a name "iot" to it



Create IoT index page (home page)

- 3) Create a file "index.html" under the [mysite/iot/templates/iot](#) folder
 - a) Right-click the "iot" folder in Visual Studio Code
 - b) Select the "New file" and assign a name "index.html" to it
- 4) Put the below HTML code (static page) to [mysite/iot/templates/iot/index.html](#) file

```
<HTML>
  <head>
    <title>IC2140 IoT</title>
  </head>
  <body>
    <h1>IC2140 IoT</h1>
  </body>
</HTML>
```

Render IoT index page (home page)

- 1) Render the "index.html" page when requested by user
 - a) Open the [mysite/iot/views.py](#) file in the Visual Studio Code
 - b) Add the following code in it

```
def index(request):  
    return render(request, 'iot/index.html')
```

Map URL to IoT index page (home page)

- 1) Create a file "urls.py" under the [mysite/iot/](#) folder
 - a) Right-click the iot folder in Visual Studio Code
 - b) Select the "New file" and assign a name "urls.py" to it
 - c) Paste the following code in it

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index),
]
```

Map URL to IoT index page (home page)

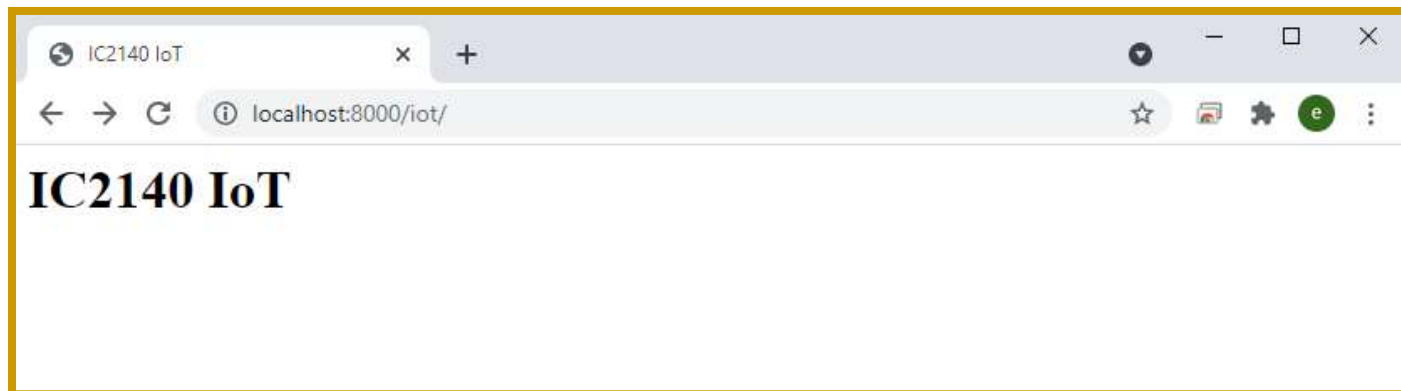
- 2) Update the routing in [mysite/mysite/urls.py](#) file (Entry point of the Project)
 - a) Open the [mysite/mysite/urls.py](#) in the Visual Studio Code
 - b) Append the "urlpatterns" variable with the following code (**Brown arrow**)



```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('blog.urls')),      #Include app "blog/urls.py"  
    path('iot/', include('iot.urls')),  #Include app "iot/urls.py"  
]
```

Access the IoT index page in browser

- 1) Start the server in the Anaconda Prompt (if stop)
\$ python manage.py runserver
- 2) Open the link as below in the Chrome Browser
<http://localhost:8000/iot/>



Create a python program to receive MQTT message

- 1) Create a file "iot_mqtt.py" under the [mysite/iot/](#) folder
 - a) Right-click the iot folder in Visual Studio Code
 - b) Select the "New file" and assign a name "iot_mqtt.py" to it
 - c) Import the packages and create some variables. Paste the following code in it

```
import paho.mqtt.client as mqtt
from .models import Event
import json

ID="11"                                # Sensor ID
mqtt_broker = "ia.ic.polyu.edu.hk"     # Broker
mqtt_port = 1883                       # Default
mqtt_qos = 1                           # Quality of Service = 1
mqtt_topic = "iot/sensor"
```

Create a python program to receive MQTT message

- 2) Create "mqtt_on_message" function to decode the received MQTT message and add the IoT data to the model "Event"

```
def mqtt_on_message(client, userdata, msg):  
    # Do something  
    d_msg = str(msg.payload.decode("utf-8"))  
    iotData = json.loads(d_msg)  
    if iotData["id"] == ID:  
        print("Received message on topic %s : %s" % (msg.topic, iotData))  
        p = Event(node_id=iotData["id"], node_loc=iotData["loc"], temp=iotData["temp"] )  
        p.save()
```

Create a python program to receive MQTT message

- 3) Create MQTT client instance, connect to broker and subscribe to the "iot/sensor" topic

```
mqtt_client = mqtt.Client("django-student_ID") # Create a Client Instance
mqtt_client.on_message = mqtt_on_message
mqtt_client.connect(mqtt_broker, mqtt_port)      # Establish a connection to a broker
print("Connect to MQTT broker")
mqtt_client.subscribe(mqtt_topic, mqtt_qos)
```

- 4) Start the loop and handle the background network tasks (thread)

```
mqtt_client.loop_start()
```

Please replace the string student_ID with your student ID

Enable the MQTT service

- 1) Open the [mysite/iot/views.py](#) file
 - a) Import the `iot_mqtt` module. Add the following code (**Brown arrow**)



```
from django.shortcuts import render
from . import iot_mqtt
```

Start the Django Server

- 1) Start the server in the Anaconda Prompt (if stop)

\$ python manage.py runserver

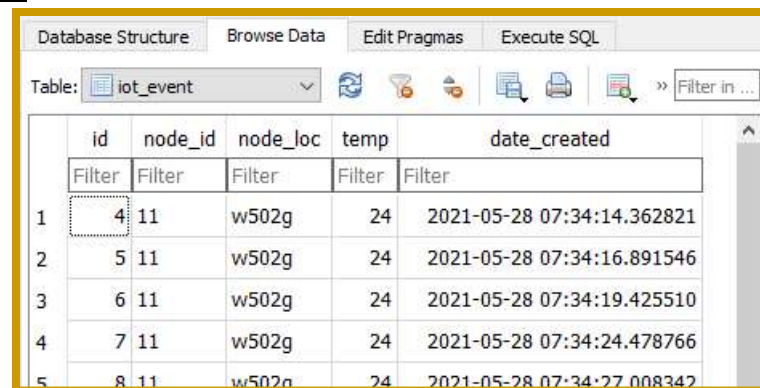
Start temperature publishing program in IoT Board

- Based on the TM1117 JSON Task 2 (Publish IoT JSON message every 2 seconds) python program in the IoT Board, append the location information in the JSON data.
- Publish the temperature, ID and location (e.g. w502g) every 2 seconds
- Execute the program for 2 mins to collect IoT data



Check the IoT data in SQLite Database Browser

- IoT board publishes the IoT Data using MQTT. The IoT data received by Django will store in the "Event" model
 - a) Open the software "SQLite Database Browser"
 - b) Click "Open Database" and select "db.sqlite3" under the [D:\01-Home\workspace\IC2140\01-Blog\mysite](#) folder
 - c) Click the "Browser Data" Tab
 - d) Select "iot_event" as the Table



The screenshot shows the SQLite Database Browser interface. The 'Browse Data' tab is active, and the 'iot_event' table is selected. The table contains five rows of data, each representing an IoT event. The columns are 'id', 'node_id', 'node_loc', 'temp', and 'date_created'. The first row has an 'id' of 4, 'node_id' of 11, 'node_loc' of 'w502g', 'temp' of 24, and 'date_created' of '2021-05-28 07:34:14.362821'. The subsequent rows follow a similar pattern with increasing 'id' values.

	id	node_id	node_loc	temp	date_created
	Filter	Filter	Filter	Filter	Filter
1	4	11	w502g	24	2021-05-28 07:34:14.362821
2	5	11	w502g	24	2021-05-28 07:34:16.891546
3	6	11	w502g	24	2021-05-28 07:34:19.425510
4	7	11	w502g	24	2021-05-28 07:34:24.478766
5	8	11	w502g	24	2021-05-28 07:34:27.008342

Extract "Event" data from database

- 1) Import the model class that we can access data "Event" in the views
 - a) Open the [mysite/iot/views.py](#) file in the Visual Studio Code
 - b) Add the 3rd statement (**Brown arrow**) in it



```
from django.shortcuts import render
from . import iot_mqtt
from .models import Event
```

Extract "Event" data from database

- 2) Retrieve all events from the database
 - a) Open the [mysite/iot/views.py](#) file in the Visual Studio Code
 - b) Update the `index()` method with the following code

```
def index(request):  
    events = Event.objects.all()  
    context = {'events' : events}      # Store the data in "context" dictionaries  
    return render(request, 'iot/index.html', context) # Pass the context to HTML template
```

Display events in the IoT index page

- 1) Insert the events in the index page
 - 1) Open the <mysite/iot/templates/index.html> file
 - 2) Place the following code after <h1>.....</h1> block

```
<table>
<tr><td>ID</td><td>Temperature</td><td>Created Date</td></tr>
{% for event in events %}
<tr>
<td><strong>{{ event.id }}</strong></td>
<td>{{ event.temp }}</td>
<td>{{ event.date_created }}</td>
</tr>
{% endfor %}
</table>
```

Display events in descending order

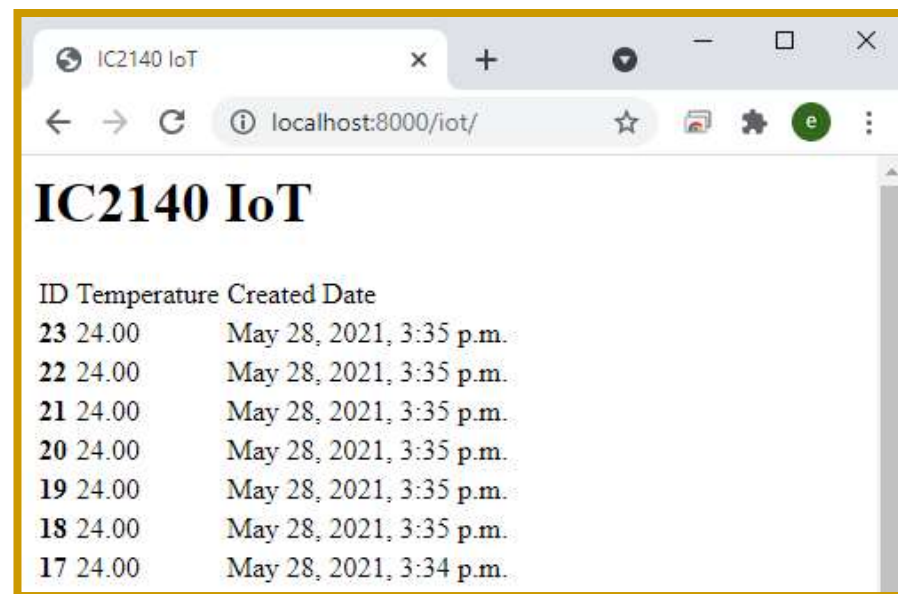
- 1) Retrieve all events from the database in descending order
 - a) Open the `mysite/iot/views.py` file in the Visual Studio Code
 - b) Search the `"Event.objects.all()"` location
 - c) Update it with the following code

```
events = Event.objects.order_by('-date_created') # date_created descending order
```


Display events in descending order

- 2) Open the below link in the Chrome Browser
(Assume that the server is running)

<http://localhost:8000/iot>



ID	Temperature	Created Date
23	24.00	May 28, 2021, 3:35 p.m.
22	24.00	May 28, 2021, 3:35 p.m.
21	24.00	May 28, 2021, 3:35 p.m.
20	24.00	May 28, 2021, 3:35 p.m.
19	24.00	May 28, 2021, 3:35 p.m.
18	24.00	May 28, 2021, 3:35 p.m.
17	24.00	May 28, 2021, 3:34 p.m.

Install Bootstrap in template

- 1) Open the [mysite/iot/templates/iot/index.html](#) file
- 2) Enable HTML5
 - 1) Add the following code (**Brown arrow**) before `<HTML>` tag



```
<!doctype html>  
<HTML>
```

- 3) Add the following code inside `<head>` section before the `<title>...</title>` tag

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-wEmeIV1mKuiNpC+IOBjI7aAzPcEZeedi5yW5f2yOq55WWLwNGmvvx4Um1vskeMj0" crossorigin="anonymous">
```

Apply Bootstrap CSS in template

- 1) Replace the `<boby>...</boby>` code block with the following code

```
<body>
  <h1 class="display-3 p-3 mb-2 bg-danger text-white">IC2140 IoT</h1>
  <table class="table table-striped table-hover">
    <thead class="table-dark">
      <tr>
        <th>ID</th>
        <th>Temperature</th>
        <th>Created Date</th>
      </tr>
    </thead>
    <tbody>
      {% for event in events %}
      <tr>
        <td><strong>{{ event.id }}</strong></td>
        <td>{{ event.temp }}</td>
        <td>{{ event.date_created }}</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</body>
```

Reference: <https://getbootstrap.com/docs/5.0/content/tables/>

Apply Bootstrap CSS in template

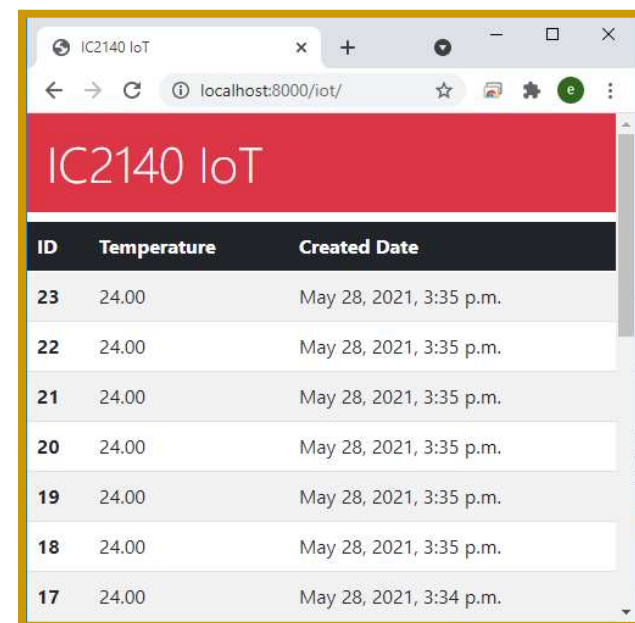
- 2) Open the below link in the Chrome Browser
(Assume that the server is running)

<http://localhost:8000/iot>



IC2140 IoT

ID	Temperature	Created Date
23	24.00	May 28, 2021, 3:35 p.m.
22	24.00	May 28, 2021, 3:35 p.m.
21	24.00	May 28, 2021, 3:35 p.m.
20	24.00	May 28, 2021, 3:35 p.m.
19	24.00	May 28, 2021, 3:35 p.m.
18	24.00	May 28, 2021, 3:35 p.m.
17	24.00	May 28, 2021, 3:34 p.m.

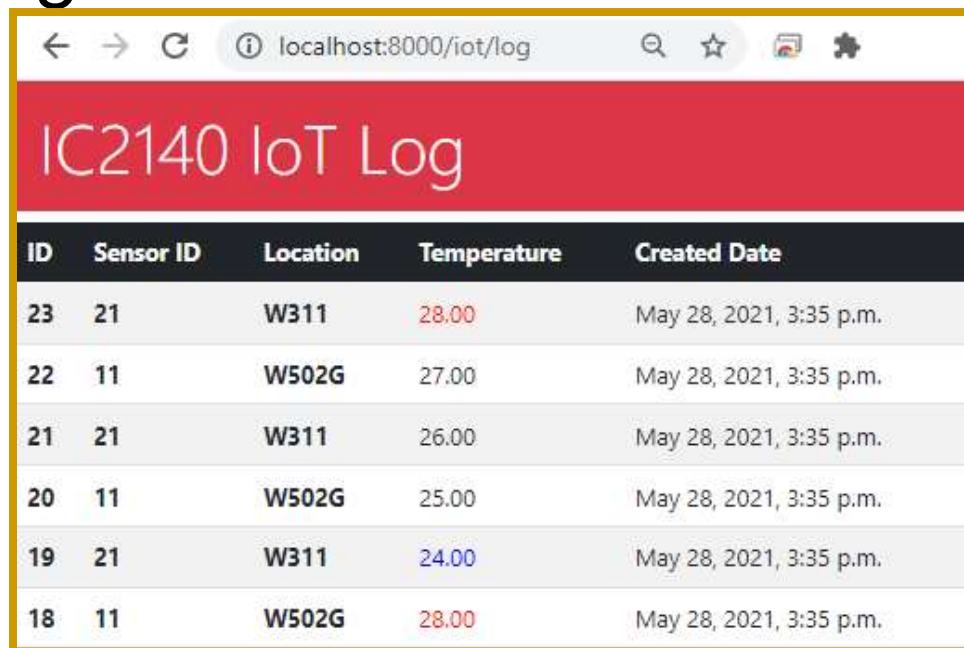


IC2140 IoT

ID	Temperature	Created Date
23	24.00	May 28, 2021, 3:35 p.m.
22	24.00	May 28, 2021, 3:35 p.m.
21	24.00	May 28, 2021, 3:35 p.m.
20	24.00	May 28, 2021, 3:35 p.m.
19	24.00	May 28, 2021, 3:35 p.m.
18	24.00	May 28, 2021, 3:35 p.m.
17	24.00	May 28, 2021, 3:34 p.m.

Django IoT Assignment 1: Django IoT sensor historical data

- Retrieve the IoT sensor historical data and display them with some conditioning formatting

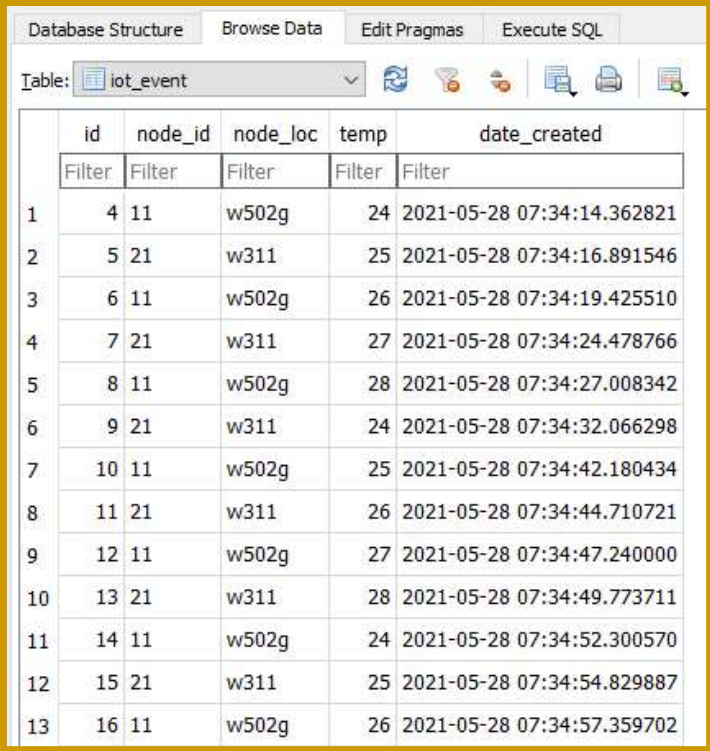


ID	Sensor ID	Location	Temperature	Created Date
23	21	W311	28.00	May 28, 2021, 3:35 p.m.
22	11	W502G	27.00	May 28, 2021, 3:35 p.m.
21	21	W311	26.00	May 28, 2021, 3:35 p.m.
20	11	W502G	25.00	May 28, 2021, 3:35 p.m.
19	21	W311	24.00	May 28, 2021, 3:35 p.m.
18	11	W502G	28.00	May 28, 2021, 3:35 p.m.

Cont.

Django IoT Assignment 1: Django IoT sensor historical data

- Assume that the database contains the following data
 - ❑ Sensor ID: 11 & 21
 - ❑ Location: w502g & w311
 - ❑ Temperature: Any value



The screenshot shows a database browser interface with tabs for 'Database Structure', 'Browse Data', 'Edit Pragmas', and 'Execute SQL'. The 'Browse Data' tab is active, showing a table named 'iot_event'. The table has five columns: 'id', 'node_id', 'node_loc', 'temp', and 'date_created'. Below the column headers are filter buttons for each column. The table contains 16 rows of data, with 'id' ranging from 1 to 16, 'node_id' alternating between 11 and 21, 'node_loc' alternating between 'w502g' and 'w311', 'temp' ranging from 24 to 28, and 'date_created' showing timestamps from 2021-05-28 07:34:14.362821 to 2021-05-28 07:34:57.359702.

	id	node_id	node_loc	temp	date_created
	Filter	Filter	Filter	Filter	Filter
1	4	11	w502g	24	2021-05-28 07:34:14.362821
2	5	21	w311	25	2021-05-28 07:34:16.891546
3	6	11	w502g	26	2021-05-28 07:34:19.425510
4	7	21	w311	27	2021-05-28 07:34:24.478766
5	8	11	w502g	28	2021-05-28 07:34:27.008342
6	9	21	w311	24	2021-05-28 07:34:32.066298
7	10	11	w502g	25	2021-05-28 07:34:42.180434
8	11	21	w311	26	2021-05-28 07:34:44.710721
9	12	11	w502g	27	2021-05-28 07:34:47.240000
10	13	21	w311	28	2021-05-28 07:34:49.773711
11	14	11	w502g	24	2021-05-28 07:34:52.300570
12	15	21	w311	25	2021-05-28 07:34:54.829887
13	16	11	w502g	26	2021-05-28 07:34:57.359702

Cont.

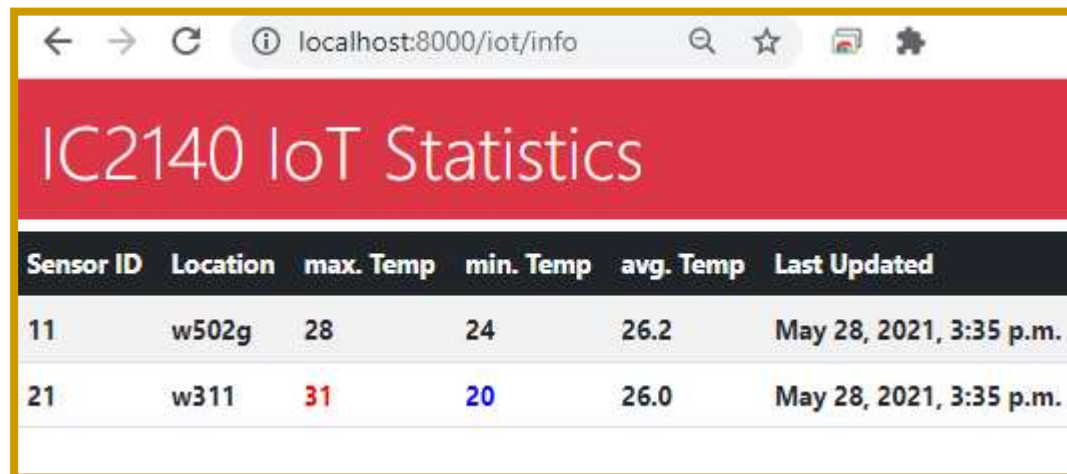
Django IoT Assignment 1: Django IoT sensor historical data

- Allow user to access it using the below link
 - <http://localhost:8000/iot/log>
- Integrate the following requirements into the example (Slide 3 to Slide 28)
 - 1) Include the Sensor ID and Location in the web page
 - 2) Indicate the warning if the temperature is higher than 29 degree with **RED** colour
 - 3) Indicate the warning if the temperature is lower than 21 degree with **BLUE** colour
 - 4) Display the location in UPPERCASE
- Show the demonstration with suggested testing

Reference: <https://docs.djangoproject.com/en/3.2/ref/templates/builtins/>

Django IoT Assignment 2: Django IoT sensor statistics

- Retrieve the IoT sensor historical data and generate the statistics
- Display the comprehensive statistics in webpage



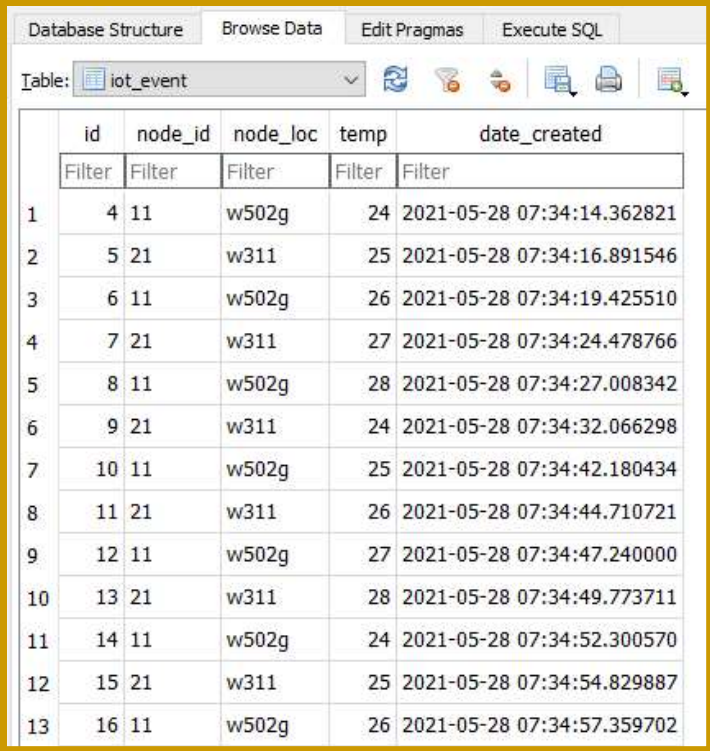
The screenshot shows a web browser window with the address bar displaying 'localhost:8000/iot/info'. The page title is 'IC2140 IoT Statistics'. Below the title is a table with the following data:

Sensor ID	Location	max. Temp	min. Temp	avg. Temp	Last Updated
11	w502g	28	24	26.2	May 28, 2021, 3:35 p.m.
21	w311	31	20	26.0	May 28, 2021, 3:35 p.m.

Cont.

Django IoT Assignment 2: Django IoT sensor statistics

- Assume that the database contains the following data
 - ❑ Sensor ID: 11 & 21
 - ❑ Location: w502g & w311
 - ❑ Temperature: Any value



The screenshot shows a database browser interface with tabs for 'Database Structure', 'Browse Data', 'Edit Pragmas', and 'Execute SQL'. The 'Browse Data' tab is active, showing a table named 'iot_event'. The table has five columns: 'id', 'node_id', 'node_loc', 'temp', and 'date_created'. Below the column headers are filter buttons for each column. The table contains 16 rows of data, indexed 1 to 13 on the left. The data alternates between node_id 11 and 21, and node_loc w502g and w311.

	id	node_id	node_loc	temp	date_created
	Filter	Filter	Filter	Filter	Filter
1	4	11	w502g	24	2021-05-28 07:34:14.362821
2	5	21	w311	25	2021-05-28 07:34:16.891546
3	6	11	w502g	26	2021-05-28 07:34:19.425510
4	7	21	w311	27	2021-05-28 07:34:24.478766
5	8	11	w502g	28	2021-05-28 07:34:27.008342
6	9	21	w311	24	2021-05-28 07:34:32.066298
7	10	11	w502g	25	2021-05-28 07:34:42.180434
8	11	21	w311	26	2021-05-28 07:34:44.710721
9	12	11	w502g	27	2021-05-28 07:34:47.240000
10	13	21	w311	28	2021-05-28 07:34:49.773711
11	14	11	w502g	24	2021-05-28 07:34:52.300570
12	15	21	w311	25	2021-05-28 07:34:54.829887
13	16	11	w502g	26	2021-05-28 07:34:57.359702

Cont.

Django IoT Assignment 2: Django IoT sensor statistics

- Allow user to access it using the below link
 - <http://localhost:8000/iot/info>
- Complete the following requirements
 - 1) List the IoT sensor data including the sensor ID and location.
 - 2) Show the summary of average, maximum, minimum of temperature in Table. Use filter() and aggregate() function
 - 3) Show the last updated of sensor
 - 4) Indicate the warning if the temperature is higher than 29 degree with RED colour
 - 5) Indicate the warning if the temperature is lower than 21 degree with BLUE colour
- Show the demonstration with suggested testing

Cont.

Reference: <https://docs.djangoproject.com/en/3.2/ref/templates/builtins/>

Django IoT Assignment 2: Django IoT sensor statistics - Hints

- Retrieve the data from the Django model using `filter()` and `aggregate()` function
- Combine the data into the context dictionary variable

```
context={'events': [ { 'id':'11', 'loc':'w502g' }, { 'id':'21', 'loc':'w311' } ]}
```

- Pass it to Django template and render a new webpage. E.g. context variable in `render()` function

```
return render(request, 'blog/blog.html', context)
```

Reference: <https://docs.djangoproject.com/en/dev/topics/db/aggregation/>